Government College of Engineering, Jalgaon

(An Autonomous Institute of Govt. of Maharashtra) Department of Computer Engineering

Student Name:_____PRN_____
Batch:_____ Date of Performance:_____Date of Completion:_____

Experiment No. 7

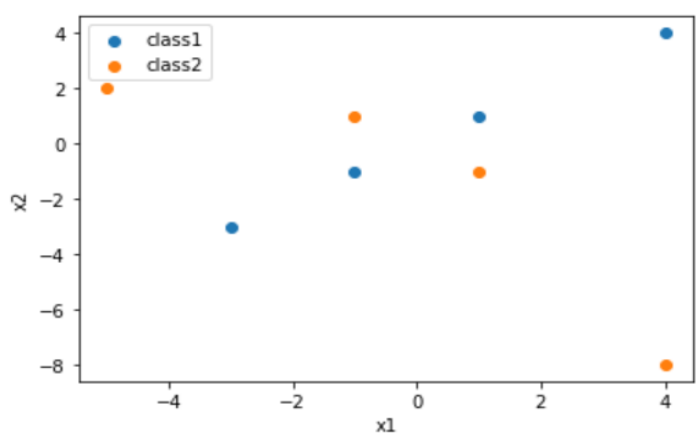
Aim: Write a program to implement classification of linearly separable data using perceptron.

Theory:

Linear Separability

Linear Separability implies the existence of a hyperplane separating the two classes. For example, consider a dataset with two features x1 and x2 in which the points (−1, −1),(1, 1),(−3, −3),(4, 4) belong to one class and (−1, 1),(1, −1),(−5, 2),(4, −8) belong to the other.

```
import matplotlib.pyplot as pltc1x1 = [-1, 1, -3, 4]
c1x2 = [-1, 1, -3, 4]
plt.scatter(c1x1, c1x2, label='class1')c2x1 = [-1, 1, -5, 4]
c2x2 = [1, -1, 2, -8]
plt.scatter(c2x1, c2x2, label='class2')plt.xlabel('x1')
plt.ylabel('x2')plt.legend()
plt.show()
```

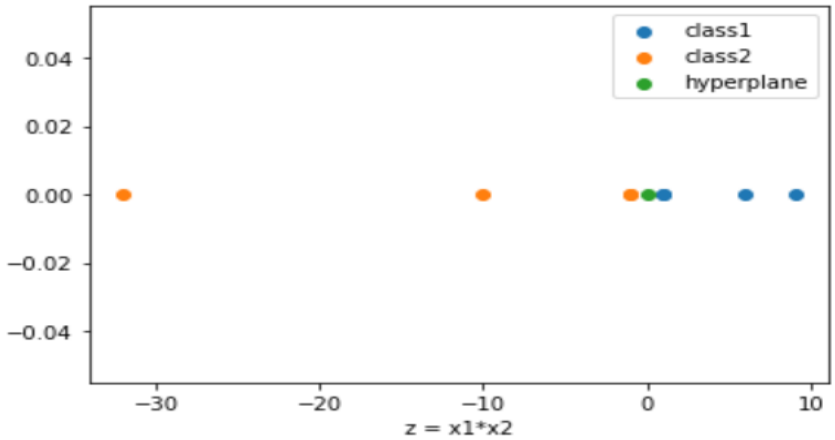


This data set is not linearly separable because there is no linear function that can separate class1 and class2.

Let's consider 1-dimensional representation z in terms of x1 and x2 such that the dataset is linearly separable in terms of 1-dimensional representation corresponding to z. Defining z = x1x2, given data can be represented 1-dimensional and be linearly separable. Using this mapping,

points of class1 (−1, −1),(1, 1),(−3, −3),(4, 4) become (1), (1), (9), (6) and points of class2 (−1, 1),(1, −1),(−5, 2),(4, −8) become (-1), (-1), (-10), (-32).

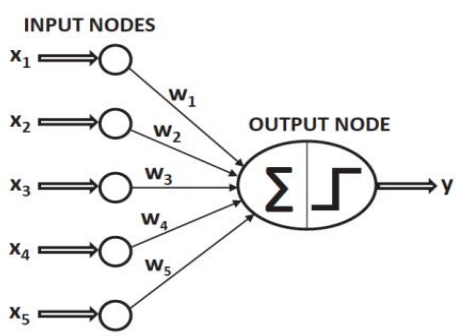
```
z1 = [1, 1, 9, 6]
y1 = [0, 0, 0, 0]
plt.scatter(z1, y1, label='class1')z2 = [-1, -1, -10, -32]
y2 = [0, 0, 0, 0]
plt.scatter(z2, y2, label='class2')plt.scatter([0], [0], label='hyperplane')
plt.xlabel('z = x1*x2')
plt.legend()
plt.show()
```



As you can see in the above plot, now z = 0 separates class1 and class2. A non-linearly separable data can be represented as linearly separable form after applying a **non-linear transformation**(z = x1x2 in this example, resulting in data represented as 1D).

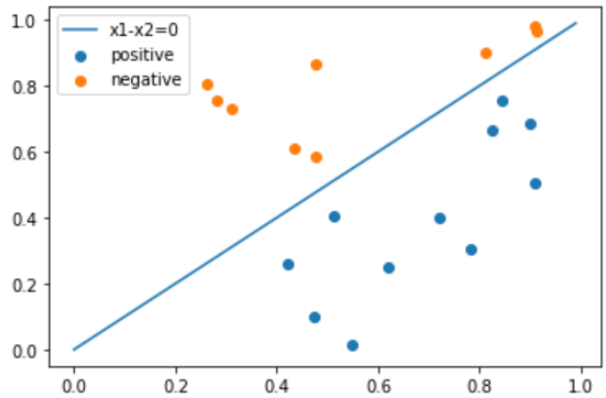
Perceptron

For linearly separable datasets, a **linear classifier or SVM with a linear kernel** can achieve 100% accuracy to classify data. Linear classifiers classify data into labels based on a linear combination of input features. A single layer perceptron is an example of a linear classifier. It computes a linear combination of input features with parameters(weights), passes it as an input for a sign function, calculates the loss, and updates the weights through gradient descent.



To better understand how a linear classifier works, let's implement a single-layer perceptron algorithm from scratch.

Consider a 2-dimensional data set in which all points with x1 > x2 belong to the positive class, and all points with x1 ≤ x2 belong to the negative class. Therefore, the true separator of the two classes is linear hyperplane (line) defined by x1 – x2 = 0. Now let's create a training data set with 20 points randomly generated inside the unit square in the positive quadrant.



Another example of a linear classifier is **SVM(support vector machines).** The objective of SVM is to find a plane that has the maximum margin, i.e the maximum distance between data points of both classes.

To evaluate the SVM on this dataset, we can change the perceptron criterion to hinge-loss and repeat the accuracy computation on the same test points above. The **hinge loss** is used for "maximum-margin" classification and equals to the following loss function we used for the above perceptron criterion(a=0) only substituting the value a to 1.

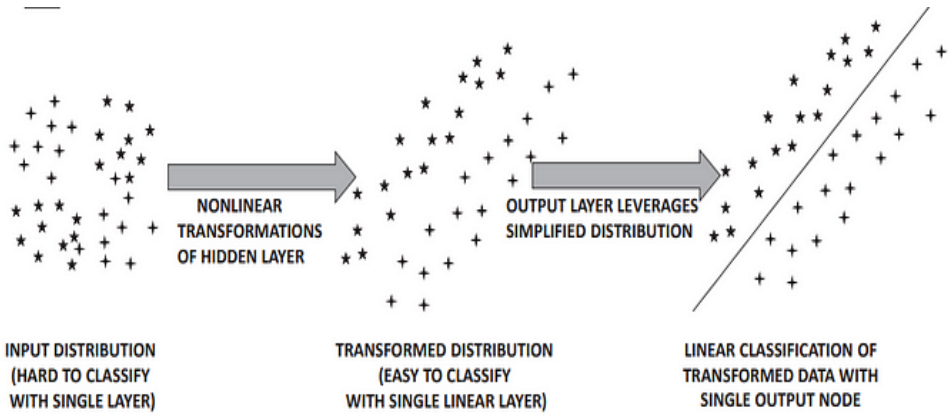
$$L = \max\{0, a - y(\bar{W} \cdot \bar{X})\}$$

```
#train
w_1 = perceptron(train_data, 10, 0.01, 1) #change a to 1
print(w_1)print("Accuracy : ", predict(test_data, w_1))
```

As in the perceptron algorithm, with SVM we could also achieve 100% accuracy on this linearly separable dataset.

Using both perceptron algorithms using linear classifier and SVM, we saw 100% accuracy can be achieved with the linearly separable dataset. However, most of the data in the real world are not linearly separable. Then how can we classify these data? Non-linear transformation is the key!

Non-linear transformation implies the key idea for multi-layer neural network classifiers. By bringing non-linear activation functions in the hidden layers, data can be represented as a linearly separable form in feature space. Thus a model can learn more complex features of data through multi-layer neural networks.



Now you may have intuition how multi-layer neural networks learn the hidden features and patterns in data. Linear classifiers are simple and work well with linearly separable data, but have limitations to learn complex features of real-world data. This is the startline to understand plenty of things about multi-layer neural network architectures to solve various real-world problems!

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score

# Generate linearly separable data
X, y = make_classification(n_samples=100, n_features=2, n_classes=2,
                           n_clusters_per_class=1, n_redundant=0,
                           n_informative=2, class_sep=1.5, random_state=42)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize the perceptron model
perceptron = Perceptron()

# Train the perceptron model
perceptron.fit(X_train, y_train)

# Make predictions on the test set
y_pred = perceptron.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f'Test Accuracy: {accuracy}')
```

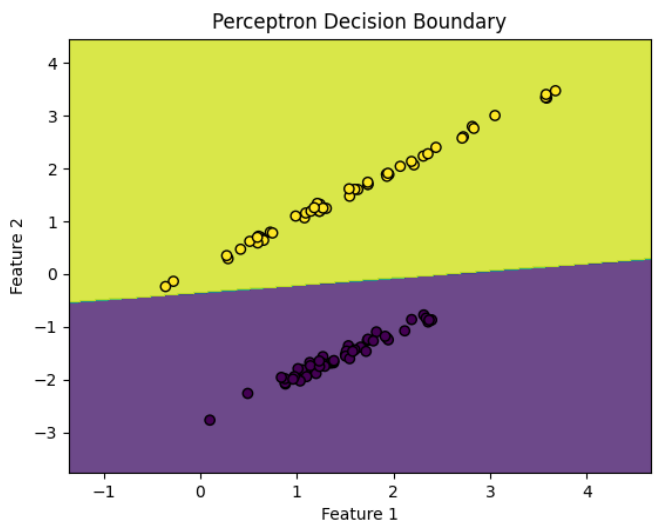
```
# Plot the decision boundary
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
                     np.arange(y_min, y_max, 0.02))

Z = perceptron.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.8)
plt.scatter(X[:, 0], X[:, 1], c=y, marker='o', edgecolors='k')
plt.title('Perceptron Decision Boundary')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

Output :

```
PS F:\Bhargav Data\Documents\8th Sem\FANN\FANNL> python .\practical_7.py
Test Accuracy: 1.0
PS F:\Bhargav Data\Documents\8th Sem\FANN\FANNL>
```



Conclusion : In this practical we learnt about implementation of classification of linearly separable data.

Ms. Diksha R. Gupta
Course Teacher